IBM POWER9 processor core

H. Q. Le
J. A. Van Norstrand
B. W. Thompto
J. E. Moreira
D. Q. Nguyen
D. Hrusecky
M. J. Genden
M. Kroener

The IBM POWER9 processor is the latest Reduced Instruction Set Computer microprocessor from IBM. POWER9 employs a new modular core microarchitecture to counter the technology trend of decreasing frequency and increasing power density from generation to generation. The new POWER9 design enables a family of processors optimized for a broad range of server applications. The new microarchitecture is closely coupled with a rich set of new instructions geared toward data-centric applications. In this paper, we describe the POWER9 core microarchitecture innovations, its new instructions and features, and the exploitation of this new design for computing in the cognitive era.

Introduction

The IBM POWER9 core continues a long tradition of delivering innovative multi-core, multi-threaded designs with a modular approach that supports a family of processor chips, with both eight and four threaded cores targeted for scale-up and scale-out products, and running the Linux, AIX, IBM i, PowerVM, and KVM operating environments [1]. POWER9 is fabricated using the GLOBALFOUNDRIES 14-nm technology with 17 layers of metal. As the silicon technology evolution slows, there is a need to improve the design of the core through microarchitecture and architecture innovation. The POWER9 core is designed to mitigate technology physical constraints by avoiding large, shared structures and by reducing the number of access ports, wiring congestion, and the exponential growth of control functions. Single thread performance is improved by keeping the processor pipeline short and distributing pipe control to local units, thus minimizing pipeline hazards. Execution units are consolidated to enable quick exchange of data and to enable mixed-data-type operations. Branch prediction capabilities are improved, and instruction control is optimized for a better out-of-the box customer experience compared with IBM POWER8. The POWER9 core builds on the strong simultaneous multi-threading (SMT) processing power of POWER8 [2] with high instruction and data bandwidth, deep out-of-order windows, and efficient cache design for high-performance throughput. Implementing the Power ISA (Instruction Set Architecture) version 3.0 [3], POWER9 implements a robust Reduced Instruction Set Computer (RISC) architecture.

Digital Object Identifier: 10.1147/JRD.2018.2854039

As shown in **Figure 1**, POWER9 introduces a modular execution slice microarchitecture, with the inner core optimized as a two-dimensional matrix. Horizontally, the inner core is built from multiple slices that are aggregated together. Vertically within each slice, major pipeline functions such as dispatch, issue, execute, and cache access are implemented. The slice is the modular building block that is highly optimized to enable a short pipeline and a low-latency execution profile. By aggregating multiple slices, we are able to provide a highly parallel core with a large pool of resources to support a high execution rate and robust hazard mitigations, providing significant per-thread performance gains, compared with POWER8, across a diverse range of workloads [4].

The POWER9 architecture supports two core variants: An SMT4 core supports up to four threads with four execution slices plus one branch slice, and an SMT8 core supports up to eight threads with eight execution slices plus two branch slices. The SMT4 core provides finer granularity for resource management by operating systems and hypervisors, including up to four logical partitions (LPARs), or one per thread. This is particularly important for cloud environments that desire a high level of resource control. The SMT8 core is optimized for larger partitions with a significant growth in core strength compared with POWER8. The SMT8 core supports live partition mobility (LPM) from POWER8 cores with eight active threads. Many transactional and data-centric business analytics applications have strong thread interaction, which can take advantage of these large partitions. In addition, innovations within the POWER9 core design are optimized for both scale-out and scale-up processors. Each POWER9 processor supports up to 96 threads, equivalent to a

© **Copyright** 2018 by International Business Machines Corporation. Copying in printed form for private use is permitted without payment of royalty provided that (1) each reproduction is done without alteration and (2) the Journal reference and IBM copyright notice are included on the first page. The title and abstract, but no other portions, of this paper may be copied by any means or distributed royalty free without further permission by computer-based and other information-service systems. Permission to republish any other portion of this paper must be obtained from the Editor.

0018-8646/18 © 2018 IBM

Authorized licensed use limited to: Jeffrey Allen. Downloaded on March 15,2021 at 00:20:01 UTC from IEEE Xplore. Restrictions apply.

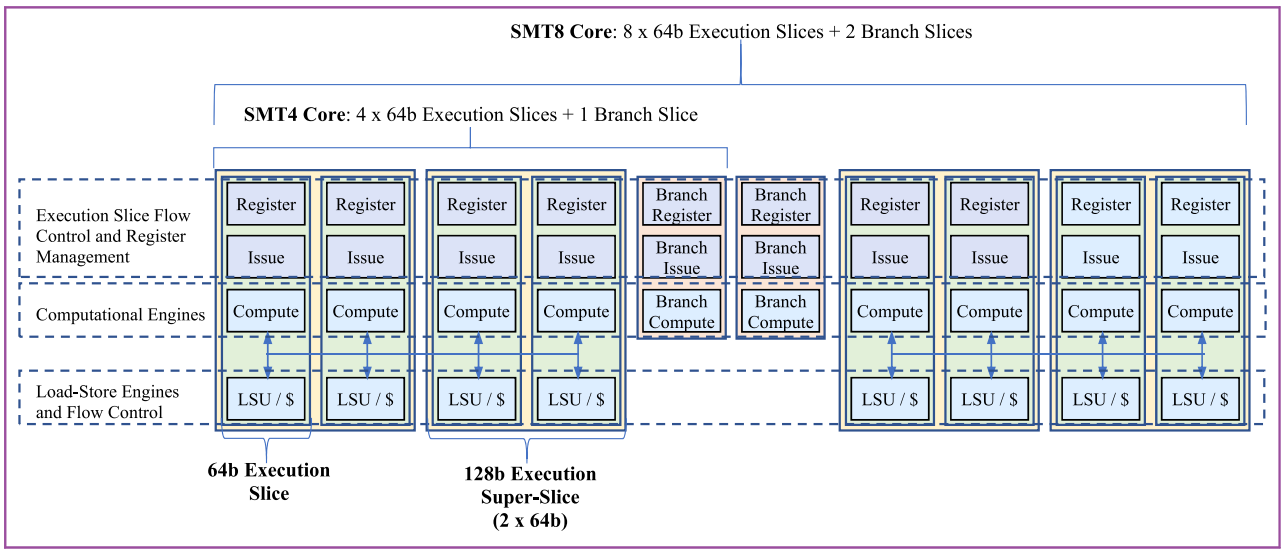


Figure 1

IBM POWER9 two-dimensional execution slice microarchitecture.

POWER8, but delivering stronger performance at various active thread counts at the chip level.

In POWER9, each execution slice has its own instruction queue, register files, execution engine, and cache port, and is implemented as a 64-bit, 8-byte double-word dataflow. Each slice can initiate a single compute operation and a single load or store operation per cycle. Issue selection, hazard detection, and recovery functions are performed within the slice. Since each slice operates on its own set of operations, physical requirements such as queue depth, number of access ports, and wiring tracks are reduced significantly. This translates into significant reduction in area and power. An example is the physical cost of tracking the age of the instructions in the queue, which reduces exponentially relative to a reduction in queue entries. Another example is the reduction of the access ports due to split (as opposed to shared) structures, which results in significant saving from area, power, and wiring congestion.

In this paper, we describe the features of the POWER9 core architecture, beginning with an overview of the core organization. We then cover key features of the instruction fetch unit (IFU), instruction sequencing unit (ISU), vector/scalar computational units (VSU), and load and store units (LSU). Finally, we highlight some of the new features of the ISA supported by the POWER9 processor.

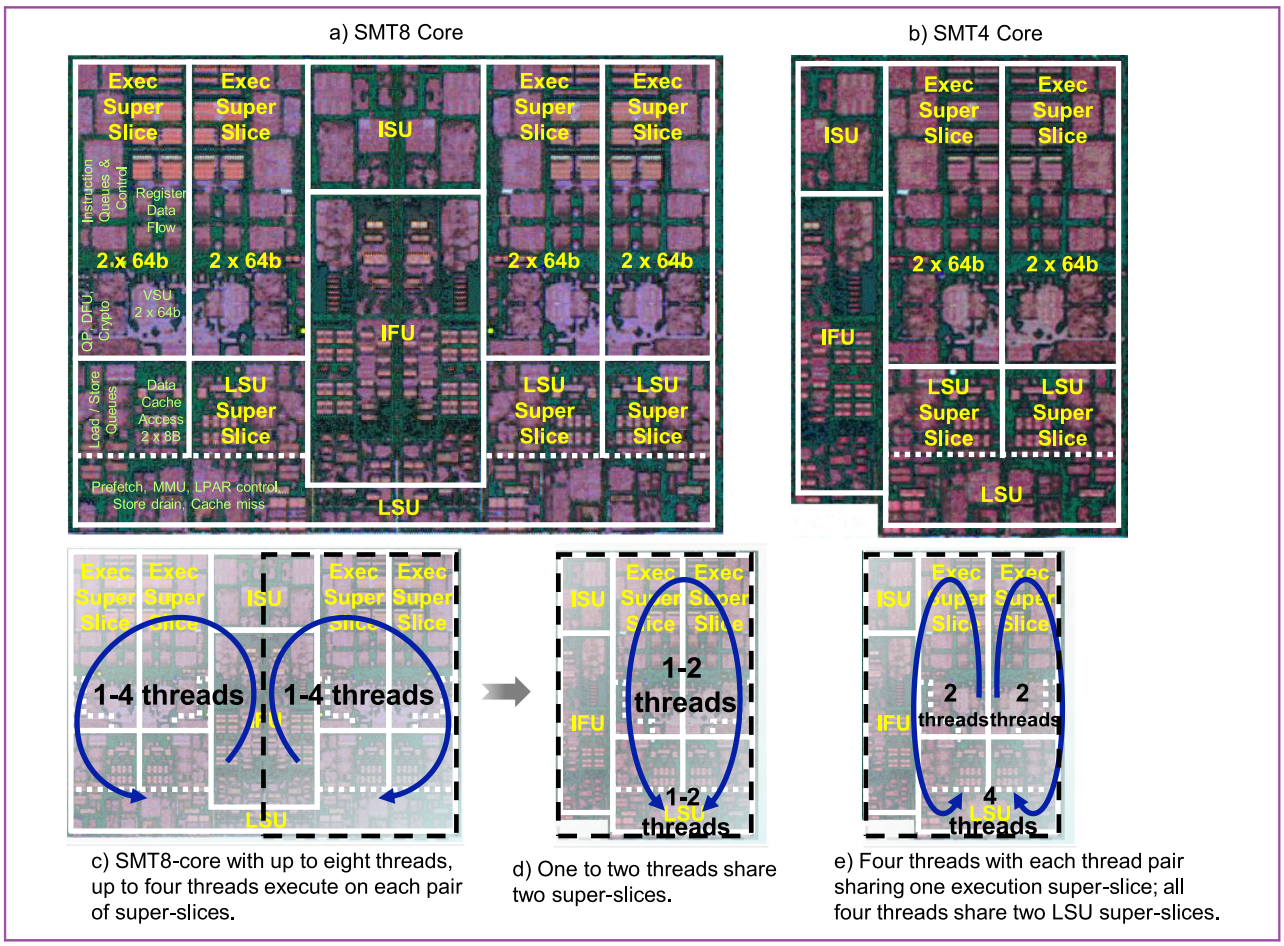
Organization of the POWER9 processor core

Figures 2(a) and **2(b)** show the physical organization of the POWER9 SMT8 and SMT4 cores, respectively. Primary elements of each core include an IFU, which contains instruction fetch and decode functions and includes the branch execution slices and the instruction cache. Decoded instructions are dispatched to the execution slices and to the ISU for insertion in an instruction completion table (ICT) where they are tracked until retirement. The execution slices manage the registers and issue of instructions to the execution engines and load-store slices within the LSU complex. The LSU manages a Level 1 (L1) data cache, access to the Level 2 (L2) cache, and also includes the memory management unit (MMU) and data prefetch functions.

The independent 64-bit double-word data-flow slice is the modular building block that allows easy construction of the inner core by joining multiple slices. A pair of slices forms a super-slice that is the physically optimized building block for the POWER9 core. Each super-slice is capable of operating on two 64-bit data flows or one 128-bit data flow. Single-instruction–multiple-data (SIMD) instructions in the POWER ISA are defined to utilize 128-bit register sources and targets and execute one operation on a super-slice on POWER9 using a coordinated execution of the two slice pipelines within the super-slice. This arrangement allows scalar codes to access a large share of the SIMD execution resources because they are equally available to both SIMD and scalar instructions. Each SMT4 core has two execution and two load-store super-slices (four double-word slices total), and each SMT8 core has four of each of the super-slices (eight double-word slices total), as seen in Figures 1 and 2.

Figures 2(c) through 2(e) show the sharing of execution resources between threads at various levels of SMT operation. POWER9 extends similar methods of resource and pipeline splitting employed on POWER8 [2] to divide the pipeline dynamically depending on how many threads

Authorized licensed use limited to: Jeffrey Allen. Downloaded on March 15,2021 at 00:20:01 UTC from IEEE Xplore. Restrictions apply.



c) SMT8-core with up to eight threads, up to four threads execute on each pair of super-slices.

d) One to two threads share two super-slices.

e) Four threads with each thread pair sharing one execution super-slice; all four threads share two LSU super-slices.

Figure 2

IBM POWER9 annotated core die photos for (a) SMT8 core and (b) SMT4 core. SMT thread partitioning shown for scenarios (c)–(e). Adapted from [4]. (Exec Super Slice: execution super-slice; QP: quad-precision float unit; DFU: decimal floating-point unit; Crypto: cryptographic unit; MMU: memory management unit; LPAR: logical partition.)

are active within the core pipeline. As shown in Figure 2(c) for an SMT8 core, the core resources are divided in half such that up to four threads can share each half of the core, the equivalent resources of an SMT4 core (hereafter called "SMT4-core resource").

A single thread spans only one half of the SMT8-core resources allowing for the other half of the core resources to be power-gated when operating in single-thread mode. Power-gating delivers a performance boost in the form of elevated frequency for the active threads on the chip. The POWER9 Workload Optimized Frequency (WOF) power management technology manages frequency boosting within a preset power budget [5]. The POWER9 architecture was conceived to alternatively support a single thread executing across all eight slices of a SMT8 core; however, this was not implemented in the final design. In this alternate design option, each SMT4-core-resource behaves as an execution cluster with additional latency incurred when dependent instructions are split between the two clusters. Performance and power analysis showed that optimal performance would be achieved for a large number of applications by instead limiting execution of a single thread to only one of the SMT4-core-resources per SMT8 core and boosting the frequency. This approach also removes the complexity associated with managing instruction routing between the execution clusters, such as choosing between utilizing both clusters to optimize for high instruction level parallelism versus utilizing a single cluster to optimize for code that is sensitive to instruction latency.

When only one to two threads are active per SMT4-core resource, each thread has full access to all execution resources, as shown in Figure 2(d). When three to four threads are active per SMT4-core resource, a pair of

Authorized licensed use limited to: Jeffrey Allen. Downloaded on March 15,2021 at 00:20:01 UTC from IEEE Xplore. Restrictions apply.

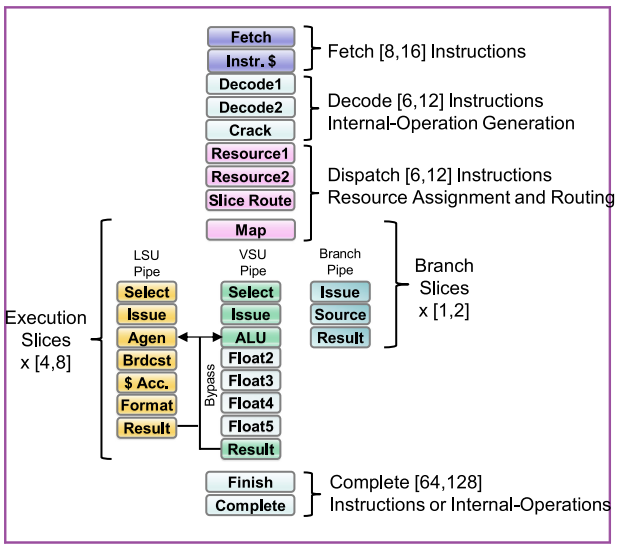


Figure 3

IBM POWER9 core instruction pipeline. Values shown as [SMT4-core-rate, SMT8-core-rate]. Instr $: instruction cache access; Agen: address generation; $ Acc.: data cache access; Brdcst: address broadcast; ALU: arithmetic logical unit and first cycle of VSU pipeline; Finish: per-pipeline finish.

threads, called a *thread-pair*, shares a single execution super-slice and all threads have full access to the two LSU super-slices, as shown in Figure 2(e).

Instructions are managed within the POWER9 core individually, removing the multi-instruction "group" management employed on many prior generations of POWER processor [2], [6]–[8]. As such, instructions may be flushed and retired individually from the processor core and resources can be freed up quickly by completing and retiring resources for up to 64 instructions per SMT4-core resource. Removal of grouping also removes two pipeline stages, contributing significantly to a shorter latency from fetch to execution, five cycles faster than POWER8. This results in a pipeline, as shown in **Figure 3**, that more closely resembles that of IBM POWER6 [9], IBM's last RISC server processor with an in-order execution microarchitecture.

Instruction issue to the execution engines and LSU slices is managed in each slice independently except for SIMD and other 128-bit operations, which require the coordination of both slices of a super-slice. Once a load or store operation is issued from the execution slice, its address determines the LSU slice on which it will operate, and the LSU slice manages further instruction flow and hazard handling. This allows the execution slice to free resources immediately after issue and allows the LSU to independently control its internal execution pipeline.

A single SMT8 core or a pair of SMT4 cores are supported by a dedicated 8-way 512-KB L2 cache and a 20-way 10-MB embedded dynamic random-access memory (eDRAM) Level 3 (L3) cache region. The L2 and L3 cache of POWER9 continues to support a non-uniform cache access (NUCA) architecture, which provides mechanisms to distribute and share cache footprints across the chip. The POWER9 L3 introduces an enhanced replacement algorithm with data-type and re-use awareness, which utilizes information from the core and L2 cache to manage cache replacement states [10]. The L2 and L3 caches support an array of prefetch requests from the core including both instruction and data prefetch with varying levels of urgency. Prefetch requests for POWER9 include additional information exchange between the core, cache, and the memory controller to manage memory bandwidth and to mitigate prefetch-based cache pollution.

Instruction fetch unit

The POWER9 IFU is responsible for bringing instructions into the core, decoding them, and transmitting them to the ISU and execution slices for downstream processing. The branch execution slices reside in the IFU to facilitate fast redirect on a branch misprediction. A fetch address generator utilizes multiple branch predictors and thread prioritization logic to enable speculatively retrieving instructions far ahead of execution. An effective address table (EAT) tracks all instruction address fetches to completion and generates addresses as required for a pipeline flush and fetch redirect or special completion event.

The instruction cache (ICache) for an SMT4-core resource is 32 KB (64 KB total split in two regions for an SMT8 core). The ICache handles both demand fetch and speculative pre-fetch requests to the L2 cache. Instructions are processed in chunks of 32 bytes (8 instructions) per cycle and are pre-decoded before entering the ICache from the L2 cache in a single stage to facilitate branch re-coding and to generate meta-instruction information used for subsequent power and timing efficient decode of instructions in the main decode pipeline. Instructions are fetched from the ICache per thread proportional to their thread priority. Fetched instructions may be stored in, or bypass, a 96-entry instruction buffer for each SMT4-core resource (192 for an SMT8-core). The instruction buffer allows the fetcher to hide redirection and speculation hazards and to service multiple threads efficiently. As shown in Figure 3, the decode pipeline produces decoded internal operations over three cycles. POWER9 reduces the number of instructions which are cracked, the operation whereby one instruction generates more than one internal operation.

Branch prediction

In addition to the direction predictors from the POWER8 core [2], POWER9 adds a local selector to the Branch

Authorized licensed use limited to: Jeffrey Allen. Downloaded on March 15,2021 at 00:20:01 UTC from IEEE Xplore. Restrictions apply.

History Table (BHT) and a TAgged GEometric history length (TAGE) branch direction predictor based on the one described by Seznec and Michaud [11]. The local selector tracks branches predicted well by the local predictor and prevents them from installing in and polluting the global predictor. The POWER9 TAGE uses four components in addition to the BHT, which acts as the base direction predictor. Each TAGE component is indexed using a different length, up to 100 bits, of a 256-bit-long global history vector (LGHV). The components are tagged and therefore only provide a prediction on a tag match, avoiding aliasing. If a tag matches in any of the TAGE components, then the prediction from the matching component using the longest length is chosen. The TAGE predictor is two cycles slower than the BHT, requiring five cycles to redirect versus three cycles for the BHT, but the TAGE predicts with greater accuracy and will override the BHT prediction if different.

POWER9 also adds a pattern-cache and re-introduces the branch target address calculator (BTAC) address predictor from POWER7 [8]. The pattern-cache is a target address predictor that can override the count-cache for branch targets that the count-cache has difficulty predicting. The pattern-cache prediction for a given branch is based on the previous resolved target of that branch. For example, it can more easily predict a branch's target that alternates between two addresses (e.g., the pattern ABABAB. . .). The BTAC provides taken branch prediction targets with a one-cycle latency for a single thread active per SMT4-core resource group (up to two active threads per SMT8-core). For correct BTAC predictions, taken branches do not waste any fetch cycles.

Instruction prefetch

Instruction prefetch performance is enhanced in POWER9 with the addition of a prefetch filter queue. Instead of pre-fetching a fixed number of cache lines according to SMT mode, the POWER9 prefetcher uses the filter queue to predict how many sequential lines to prefetch. The filter queue tracks recent ICache misses along with an indication of whether cache lines sequentially following the miss are eventually requested. This information gets saved to a backing array that later determines how many cache lines to prefetch the next time the address misses in the cache. The POWER9 IFU can prefetch from zero (prefetch suppression) up to seven cache lines ahead when there is one thread active per SMT4-core resource group, and from zero to three cache lines ahead where there are two threads active per SMT4-core resource group.

Instruction sequencing unit

The ISU manages instruction execution resources and flow control. A dispatcher routes instructions and/or internal operations (hereafter referred to simply as "instructions") to an execution slice based on resource availability, thread assignment, and heuristic-based policies to avoid resource contention. The POWER9 dispatcher may dispatch a subset of the available instructions presented by the decoder in a given cycle and is able to maintain flow control such that instructions are presented to execution slices with available resources each cycle. As depicted in Figure 3, the decoder and dispatch handle up to six instructions per SMT4-core and up to 12 instructions per SMT8-core per cycle. When more than two threads are active per SMT4-core resource, the decode and dispatch pipes are split such that two thread-pairs can each process three instructions per cycle. In an SMT8-core, four thread-pairs can process three instructions each.

The dispatcher performs resource checking and routing over the course of three processor clock cycles as depicted in Figure 3. The dispatcher also performs dependency checking functions for registers and for storage operands. Storage operand dependency identification has been enhanced for POWER9 to avoid out-of-order storage accesses such as executing a younger load before an older store resulting in a store-hit-load (SHL) flush. The enhanced mechanisms include both a proactive dependency identification based on instruction operands and an enhanced reactive dependency identification based on historical occurrences of SHL. An execution slice performs its own instruction flow control and resource management for each instruction received from the dispatcher.

Each execution slice handles the register renaming function (mapping) and handles backing up register state to facilitate flush restore when needed. Within a super-slice, some optimizations such as port sharing occur to maximize the utility of the physical implementation. Each super-slice can receive up to three instructions dispatched per cycle, for a total of six per SMT4-core and 12 per SMT8-core per cycle. The renaming functions are performed on groups of registers that share a renaming name-space, physical register resources, control, and data-flow across the core. The general-purpose registers (GPRs) and vector scalar and floating point registers (VSRs) constitute one such group, thus enabling native exchange of data and interchange of operands between the GPRs and VSRs. The Floating Point Status and Control Register (FPSCR), Fixed Point Exception Register (XER), Condition Registers (CRs), and Vector Status and Control Register (VSRC) constitute a second such group. Both groups utilize a history-buffer for flush restore and a common renaming mechanism. The Link, Count, and Target Address Register (TAR) constitute a third group that utilizes a non-history-buffer-based mapper with 20 entries per SMT4-core resource to perform rename and flush restore functions.

When renaming in a history buffer, each register target is assigned a history buffer location that is used to reference older register data. In the case of a flush event, the

Authorized licensed use limited to: Jeffrey Allen. Downloaded on March 15,2021 at 00:20:01 UTC from IEEE Xplore. Restrictions apply.

instruction tag (ITAG) and the older source data in the history buffer need to be restored into the mapper and register file state. To save power, two levels of history buffer, a level one (L1HB) and a level two (L2HB), are used per slice. The L1HB contain entries for instructions that are in-flight but have not written back. The L2HB history buffers contain entries that have not yet completed but have received all their data. The L2HB also contains the pre-transactional value of registers modified during transactional memory operation. Each GPR/VSR history buffer has 20 entries in L1HB and 96 entries in L2HB per slice. Each history buffer entry can track data for a scalar target register, and two entries are used to track data for a vector target register (VSX/VMX register). The FPSCR/XER/CR history buffers track up to four register fields. Each FPSCR/XER/CR history buffer has 12 entries in L1HB and 12 entries in L2HB per slice.

The ISU employs an ICT to track all in-flight instructions after dispatch. The ICT has 256 entries per SMT4-core resource that are statically shared by all active threads. Each ICT entry corresponds to at most one instruction. ICT entries are partitioned based on the number of active threads. When all threads are active, each thread has 64 ICT entries.

Each execution slice manages instruction flow including instruction issue, cancel and retry, and pipeline contention. An issue queue in each slice holds up to 13 instructions for a total of 54 instructions per SMT4 core and 108 instructions per SMT8 core. Each execution slice issue queue can select one load or store address generation operation, and one computation operation per cycle to issue (up to two instructions issued per cycle per execution slice). Instructions are selected based on age order, operand availability, and pipeline availability for shared or multi-cycle operation pipelines. Different from recent generations of POWER designs, once a load or store instruction issues with correct operands (i.e., once it can produce a valid address), the operation is released from the issue queue and is managed by the load-store unit associated queues and LSU slice control. This grows the effective size of the main issue queues since they do not need to hold onto load and store instructions that have not yet cleared pipeline hazards, hit the cache, or resolved address translation.

Each branch slice (two in total per SMT8 core) can receive up to two branch instructions dispatched per cycle from an associated group of six decode lanes. Each branch slice also includes an issue queue, each of which holds up to 16 instructions. In total, the POWER9 issue queues can issue up to nine instructions per SMT4-core and up to 18 instructions per SMT8 core per cycle.

Vector/scalar computation units

The basic element of the VSU is the double-word slice which supports execution of a single instruction with up to three 64-bit operands, an auxiliary operand (FPSCR, XER, CR, VSCR) and a 64-bit target register plus an auxiliary target register (FPSCR, XER, CR, VSCR). This basic element includes a heterogeneous mix of computational pipelines including support for both fixed-point and floating-point operations. To provide seamless operations between different data types and to improve area efficiency, the fixed-point unit is integrated into the VSU. The fixed-point and the floating-point multiplication operations share the same multiplier logic. The latency of the floating-point operations is reduced in comparison to its predecessor [2] and is now at five cycles issue to issue as shown in Figure 3. The shorter floating-point pipeline also decreases the latency for floating-point divide instructions by 15%. When a dependent floating point operation is in a different super-slice from the producer, an additional two cycles are incurred in the issue to issue latency.

At the physical optimization level of the super-slice, a permute pipeline, a fixed-point divide pipeline, and a new decimal/quad-precision fixed point pipeline are added. The fixed-point division engine is improved and supports a faster result for many cases based on a new “early out” mechanism.

Each pair of super-slices (a single SMT4-core resource) shares a crypto unit and a decimal floating-point pipeline. The decimal floating-point pipeline from POWER8, which already supports quad-precision decimal floating point instructions, is extended in POWER9 to include native support for fully IEEE 754-2008 compliant binary 128-bit floating-point arithmetic.

Load and store unit

The POWER9 LSU is arranged as a collection of address-based Data Cache (DCache) slices that perform load and store operations and share access to an L2 cache, MMU, and data pre-fetcher. The DCache for an SMT4-core resource is 32 KB in capacity, accessible by four address-based cache ports. For an SMT8 core, the DCache has a total capacity of 64 KB, split in two regions, each shared by up to four threads. Each cache port provides access to one quarter of the 32-KB DCache region based on the double-word (8-byte) address offset. That is, each consecutive 8 bytes of storage is accessed by the next logical cache port modulo four. For example, an access to a 16-byte storage access naturally aligned to an 8-byte boundary, such as from a 128-bit vector load instruction, will utilize two consecutive cache ports to perform the access.

The execution pipeline for a load (or store) instruction on POWER9 is shown in Figure 3. As a by-product of address generation, the target slice(s) for an instruction become known and the appropriate slices snoop the operation in the broadcast (“Brdcst”) cycle. In the following cycles, the

Authorized licensed use limited to: Jeffrey Allen. Downloaded on March 15,2021 at 00:20:01 UTC from IEEE Xplore. Restrictions apply.

DCache is accessed, and data is formatted and then bypassed to a dependent operation with an issue to issue latency of four cycles.

As with the execution slices, pairs of LSU slices are also physically optimized as a block called an LSU super-slice, with two super-slices per SMT4-core resource. Some structures and ports have been optimized at the super-slice granularity, while others are built at a slice granularity.

Previous LSU designs had symmetric cache access ports that could act unilaterally on any or all loads or stores or both. This kind of accessibility comes at a price, specifically the time it takes to traverse a full cache to access data or the time it takes to detect a load-hit-store (LHS) hazard and subsequently forward from older store data to a younger load result. The POWER9 address sliced design provides access to only one quarter of the full cache capacity per port, and similarly each companion portion of the load and store tracking structures only tracks, on average, one quarter of the data accesses. The store-reorder-queue (SRQ) tracks a double-word of data per slice, and the load-reorder-queue (LRQ) is unified across the super-slice. Therefore, although it tracks double-word granularity data per slice, it can also track up to a quad-word (16 bytes) of data within a single entry as a physical optimization. Thus, a load access on POWER9 can be accomplished relatively quickly, whereby the process to locate data within the L1 cache, read it out, and format it fully to be ready for write back delivery to the resultant GPR or VSR requires only one and one quarter clock cycles.

Likewise, due to the smaller number of entries in any one SRQ, any older store that is detected as an LHS hazard can be quickly identified, read out of the SRQ data structure, and multiplexed into the write back path. POWER9 supports store data forwarding for LHS at only two cycles longer than the data cache read; this path could have been faster but was optimized for aligning the wakeup of dependent instructions with the execution slices. An LHS hazard on POWER9 results in only two cycles of wasted speculative issues from the execution slices, a significant improvement over previous designs. The full complement of SRQ store entries is available for such forwarding. POWER9's SRQ can also forward entries that are not fully contained in the queue and require a merge between an SRQ entry and the background data from the DCache, whereas previous designs only supported fully contained forwarding from the SRQ with no missing bytes. With the sliced design, this mix of store data and/or background data can occur independently for each double-word slice.

POWER9 further improves upon previous designs on the LHS protection for avoiding false positives. POWER8 and prior designs matched the effective address (EA) bits 48:63 between the younger load and the older store queue entry. In POWER9, through a combination of outright matches for EA bits 32:63 and hashed EA matches for bits 0:31, false positive avoidance is greatly improved. This reduces the number of flushes, which are compulsory for false positives.

Single load instructions can span multiple double-word slices in at least two ways. First, from the earlier example, an instruction accessing 16 bytes necessarily spans at least two slices. The second way is, from any instruction that loads or stores larger than a single byte and crosses a double-word boundary. Any double-word boundary may be crossed, including cache-line boundaries or a translation page boundary, and the POWER9 LSU system will handle it without the intervention of any flushes or microcode. When this occurs, the two or three slices involved independently execute the operation. Thus, for a load, the disparate slices may produce their results at different times, under different conditions. For example, Slice A may get a clean L1 DCache hit, while Slice B encounters an LHS hazard. There may be a slice C that crosses a translation boundary and may need to utilize the MMU for address translation before it can proceed. In any case, the slices involved each produce an independent local result as fast possible and deliver these partial results back to the target GPR/VSR. The target register is given the boundary alignment control from the LSU slice at result write-back time, and the result will be assembled in the register file. An unaligned access that crosses a 128-byte cache-line boundary is handled within the LSU slice by a local increment of the address. As each slice completes processing of its portion of an operation, it generates a finish. The finishes are aggregated in the ICT to determine when an instruction is eligible to complete.

LSU slice flow control

Execution slices issue load and store instructions without a priori knowledge of their LSU slice number since address computation has not yet taken place as described above. As a result, pipeline contention may occur if more than one instruction is issued to the same LSU slice at the same time and/or if the LRQ or SRQ of a slice is fully utilized. These contention conditions are detected in the broadcast cycle and contended instructions are queued per LSU super-slice in a load-store-address queue (LSAQ). The LSAQ serves as a local collector and supports speed-matching instruction flow control into the slices. Working in tandem with the LSAQ is a local collector in the VSU to buffer store data and addresses to line up delivery for store operations. The LSAQ also serves to enable an alternate use of the per-slice access ports by allowing for a load and store operation of up to 16 bytes each to execute concurrently on the ports of each LSU super-slice.

An additional implication of the execution time assignment of instructions to LSU slices is that LRQ and SRQ resources cannot easily be determined in advance and must therefore be self-managed. In some circumstances, speculative out-of-order operations may result in a potential

Authorized licensed use limited to: Jeffrey Allen. Downloaded on March 15,2021 at 00:20:01 UTC from IEEE Xplore. Restrictions apply.

deadlock for queue resources if care is not taken. A traditional way to mitigate these risks in a self-managed queue would be to reserve an entry for each active thread to ensure forward progress for the oldest instruction on each thread can be made. However, this method is expensive with a high degree of SMT such as on POWER9 where up to four threads can share access to each LSU sliced queue. The POWER9 core therefore introduces a method of "most favored instruction-tag" (MFI) where the ISU manages a single oldest favored instruction for each group of up to four threads. Queueing structures and points of flow control implement either bypasses for the MFI instruction or reserve a single queue entry to allow the MFI to make progress. The ISU ensures that each thread gets a turn as MFI to guarantee progress.

Previous POWER designs required that the ISU manage instruction execution and pipeline flow through the life of the LSU instruction. This required holding pending instructions that encounters hazards, including cache miss or MMU processing, within the primary issue queues. In contrast, each LSU slice of POWER9 manages its local pipeline including any re-entry into the LSU pipeline. With this approach, the execution slice issue queue is able to release the LSU operation immediately after a successful address generation, and the LSU is able to optimally manage its internal pipeline by avoiding artificial pipeline collisions between incoming and re-circulating instructions. The control of instruction re-issue in the LSU slice is more refined in POWER9, when compared to previous designs. The LSU only re-issues at the exact time that it is deemed most ideal, based on the source of a hazard or resource contention. In contrast, on previous machines, in many cases, the primary issue queue would re-issue an instruction as soon as possible to help hide the long wake-up time and the chance of missing the first possible successful execution cycle. Often, this approach could result in re-issuing an instruction again and again, wasting power and execution bandwidth. Some machines have attempted to provide guidance related to outstanding translation or long latency memory fetches to instruction wakeup but still maintain instructions in the primary issue queue. However, these methods are often coarse-grained and may require longer lead time decisions in order to cover the communication delay between the LSU and the primary issue queues.

Scale-up and scale-out optimizations

The POWER9 family includes processors that support both direct attached and buffered memory systems. Each chip provides robust memory bandwidth capabilities with direct attached memory chips providing eight DDR4 channels. The POWER9 processor core includes optimizations for each type of system including those with a reduced number of memory channels. The data prefetcher has been optimized to perform optimally on both types of systems by modulating the prefetch depth and policy for each system type as well as dynamically based on feedback from the memory controller. The dynamic controls include mechanisms to balance memory bandwidth saturation avoidance and data streaming requirements. The core works in concert with capabilities in the memory controller and L3 caches to achieve this balance. This is especially important in systems with only four channels of direct attached DD4 memory.

The base-size cache-line in the POWER9 system is 128 bytes. POWER9 systems with direct attach memory, under the control of the heuristics in the processor core and memory hierarchy, can reduce the size of the data granule being fetched per cache-line to 64 bytes under high memory bandwidth contention. This mechanism can provide up to double the effective memory bandwidth for applications with sparse data accesses, such as graph-based workloads in a scale-out configuration. The cache hierarchy, including the DCache, mark partial lines in the directories and fetch the missing portion of the line if needed.

Architectural exploitation for emerging applications

POWER9 introduces many new architectural features and instructions targeted at a broad range of applications. Traditional enterprise applications have focused on the transformation of business data through well-defined processes and algorithms. An example includes bank account management, in which the balance of the account must be updated as each deposit or withdrawal is processed. While these traditional applications are vital for the health of the economy and for the success of an enterprise server computer, a new generation of enterprise applications is emerging. These new applications generate value by extracting knowledge and insight from the business data and transforming that knowledge into business actions. An example includes purchase recommendation systems, in which data about products and consumers are used to drive sales.

These modern enterprise applications have significant differences from their more traditional counterparts. The business data used as input may not be nicely formatted and tabulated. More often than not, unstructured data needs to be processed. Furthermore, that data go well beyond just numbers and strings and include voice, video, images, scanned text, PDF documents, just to name a few. Even after that, data is digested and transformed into numbers, a variety of floating-point, integer, and decimal formats may be used.

The processing of data itself is likely to require various analytics techniques, from statistical methods to deep neural networks. These techniques can really drive up the demand for computing, particularly as more elaborate computations deliver better results including accelerated solutions. Complicating matters further, techniques can be

Authorized licensed use limited to: Jeffrey Allen. Downloaded on March 15,2021 at 00:20:01 UTC from IEEE Xplore. Restrictions apply.

composed dynamically, so even the structure of the computation is not known in advance. These dynamic computations demand robust computation with low latency and high-bandwidth connections to enable quick data exchange, such as between processing cores and/or accelerated computation engines. In addition, many of the tools used for analysis are built on scripting languages such as Python and R.

Finally, security is of paramount and increasing concern to enterprise applications. Data has to be protected and certified before being transmitted and used. This drives the demand for computation even further, through cryptography and other provenance mechanisms like Blockchain.

The POWER9 processor core was designed to tackle the demanding environment of modern enterprise applications. As we will see, its architecture has allowed us to expose a diverse set of new primitives, in the form of Power ISA instructions, that we can use to efficiently execute the various computations mentioned above. Our goal is to sustain the growth of these emerging applications through a processor core that can deliver the necessary computing performance at all levels of the enterprise application stack.

Many new instructions and architectural features are introduced in POWER9 such as modulo operations to provide support for in-core hashing, and fixed-point multiply-add instructions to accelerate Blockchain's implementation of elliptic-curve cryptography (ECC) digital signature algorithm.

Floating-point formats

IBM POWER processors have a tradition of strong support for floating-point computations. Modern enterprise applications include complex computations that, in addition to the traditional single- (32-bit) and double-precision (64-bit) formats, also require half- (16-bit) and quad-precision (128-bit) floating-point formats. POWER9 is the first POWER processor with native support for both 16- and 128-bit floating-point numbers. The 128-bit numbers offer vastly enhanced precision, whereas 16-bit numbers allow more compact encoding, improving both cache behavior and effective memory bandwidth.

Support for 128-bit quad-precision floating-point computations takes the form of a complete set of arithmetic (add, subtract, multiply, multiply-add/subtract, divide, absolute value, negate, square root), logic (comparison), and conversion (to and from other floating-point and integer formats, including rounding) instructions. These instructions offer hardware acceleration for scientific algorithms that require a high degree of precision. In comparison, the IEEE 16-bit floating-point format is intended to be a storage format only. Therefore, POWER9 provides instructions for converting data between the 16-bit half-precision format and the more conventional single- and double-precision formats.

Both 16- and 128-bit floating-point numbers are stored in the 64×128-bit VSRs. 128-bit numbers are always treated as scalars, with one number per register. 16-bit numbers can be treated either as scalars (one number per register) or vectors (8 numbers per register). Either way, the large register space on the VSR allows programs to keep enough state in fast registers to keep the functional units busy.

Still leveraging the large size of the VSR, POWER9 introduces several new floating-point instructions for numbers in both single- and double-precision formats stored in the VSR. These instructions support various comparisons and selections of floating-point values, as well as extraction, insertion, and comparison of just the exponent or mantissa fields.

Atomic memory operations

The POWER9 processor has been augmented with a new set of instructions that can atomically update memory and are executed at the memory controller. These instructions have the general fetch-and-operate form: A value is read from memory, modified using a particular operation, and stored back in the same memory location. Operations include the usual arithmetic (add, max, min) and logic (and, or, xor) primitives, as well as swapping (and conditional swapping) of values with a register. The instructions come in two variants: The store variant updates only the memory location, whereas the load variant also returns the resulting value in a register.

Atomic memory operations are key for the fast execution of synchronizations and parallelization of fine-grain constructs. Common use in analytic applications include the evaluation of reductions as well as the computation of histograms. Traditional locking mechanisms can significantly jeopardize the performance of these operations on a highly parallel processor such as POWER9.

Acceleration enablement

POWER9 processors have exceptional accelerator capabilities including the NVLink interface [12] as well as for accelerators attached over the CAPI and OpenCAPI interfaces [13] and on-chip accelerators. To fully leverage the power of theses accelerators, the communication between the CPU and the accelerator as well as other chips in the system must have high-bandwidth and low latency as well as coherent data exchange capabilities.

The POWER9 processor implements the copy/paste pair of instructions. When executed by the processor, a copy instruction loads a full 128-byte cache line from a memory location, whereas the paste instruction stores that data to a different memory location and can communicate success for contended or conditional scenarios. These instructions can be used to quickly copy data between memory local to

Authorized licensed use limited to: Jeffrey Allen. Downloaded on March 15,2021 at 00:20:01 UTC from IEEE Xplore. Restrictions apply.

the CPU and other targets within the system and across the SMP fabric. These instructions also serve to enable direct user-mode access to on-chip accelerators [10].

As emerging cognitive applications including high-performance analytics grow in use, it is becoming more common to use attached accelerators as part of a server solution. This has become particularly common in the case of deep learning applications, where attached GPUs provide the bulk of the computing power necessary for these demanding computations. For these and other emerging accelerated applications, POWER9 fully integrates support for the efficient coherent exchange of information with accelerators. The processor core MMU was architected in concert with a nest-MMU (NMMU) to seamlessly facilitate coherent data access across the accelerator interfaces. The enablement of coherent data exchange between the processor cores and accelerators lowers the overhead and effective latency for data exchange and reduces the programming overhead for managing these latencies in applications with dynamic exploitation of various analytic techniques. Both MMU designs introduce support for the POWER radix page table architecture. This address translation structure, found in various other processors, is a natural match to the way the Linux operating system manages memory, and was designed to provide optimal performance for typical applications under that operating environment. The traditional hashed page table, used by prior POWER processor designs, continues to be fully supported in POWER9.

Little-endian processing

Most attached accelerators use a little-endian memory model. The little-endian support, already present for several generations of POWER processors, was significantly enhanced in POWER9.

Because little- to big-endian conversion has to be performed on a per-element basis, various new vector instructions were added. These include load and store vectors of byte (8-bit) and half-word (16-bit) elements, as well as byte-reversal within a vector register (VSR) on half-word, word, double-word, and quad-word elements. Various insert and extract element instructions, operating on the VSR, were also added.

All these instructions further support efficient operation of the POWER9 processor in little-endian mode, leading to a seamless integration with attached accelerators. This is particularly important given the support for a unified memory space between CPUs and GPUs in POWER9 Systems.

Secure computing

Several features and instructions have been added to the POWER9 processing core to support general secure computing. One of the building blocks of secure computing is arbitrary precision arithmetic, in which standard arithmetic operations (add, subtract, multiply, divide) are performed in very large integers of hundreds and even thousands of digits. Arbitrary precision arithmetic is crucial for the computation of asymmetric cryptographic keys, and is supported, for example, through the GNU Multiple Precision Arithmetic Library (GMP).

Among the various instructions added to POWER9 to speedup arbitrary precision arithmetic are three variations of 64-bit integer multiply-add instructions (maddhd, maddhdu, and maddld). These instructions leverage the unified 64-bit multiply-add circuit in the POWER9 microarchitecture, which can work on both floating-point and integer data. Instructions that compute the higher half of the result (maddhd, maddhdu) can be combined with the maddld instruction, which computes the lower half of the result to produce a full 128-bit result. Multiply-add of wider integers is then supported in software by combining several of these instructions.

Still in the scope of multiply-add operations, a new vector instruction (vmsumudm) computes the dot-product of two vectors of 64-bit integers, accumulating it with a 128-bit scalar. Previously, the Power ISA only included dot-product operations on 8- and 16-bit data, with 32-bit accumulators.

Also improving the performance of arbitrary precision arithmetic, POWER9 has an instruction (addex) to perform add-with-carry using the overflow bit (OV) as the carry. This supports the summation of multiple independent chains of terms, one using the traditional carry bit (CA) and the other using OV.

Another important operation for security is the computation of hash functions. To accelerate such computations, POWER9 has added several instructions to count trailing zeros in data, in both scalar (in GPR registers) and vector (in the VSRs) form. These new instructions complement the functionality of counting leading zeros, introduced in POWER8.

Also in support of hashing operations, the POWER9 ISA includes now a full set of modulo instructions to directly compute the remainder of an integer division. Previously, computing the remainder required computation of the quotient, through a regular divide instruction, followed by multiplication and subtraction operations. The new instructions operate on both word (32-bit) and double-word (64-bit) integers and perform both signed and unsigned arithmetic.

Finally, the POWER9 processor includes a true random number generator with guaranteed lower bound on its entropy. True random numbers have important applications in security. An executing program can read a new random number using the *darn* (Deliver A Random Number) instruction.

Decimal arithmetic

Decimal data types, both integer and floating-point, are of great importance in enterprise computing. Currency, in

Authorized licensed use limited to: Jeffrey Allen. Downloaded on March 15,2021 at 00:20:01 UTC from IEEE Xplore. Restrictions apply.

particular, must be represented as a decimal data type in order to guarantee exact accounting. The Power ISA has traditionally included various instructions for decimal arithmetic, and POWER9 has added several new ones.

Most new decimal instructions in POWER9 manipulate large integer decimal numbers, stored in the VSR as scalar data. The instructions perform various decimal arithmetic operations and convert between different decimal formats and between decimal and binary coded decimal formats. These instructions are important for business analytic applications such as those used in columnar and in-memory databases. There are also new instructions to multiply binary data by 10 and to load and store decimal data from and to memory. Finally, there are a couple of new instructions to manipulate decimal floating-point data, stored in the usual FPR registers.

Vector FXU operations

The POWER9 core microarchitecture uses a unified fixed-point unit (FXU) to process both scalar (GPR) and vector (VSR) data. With that, it is natural to enhance the vector instruction set to perform the same fixed-point operations already supported in the scalar instruction set. The new vector instructions include various element-wise comparison, sign extension, arithmetic, rotate, shift, and parity operations. Element widths supported include byte (8-bit), half-word (16-bit), word (32-bit), and double-word (64-bit), although not all operations support all the element widths.

Conclusion

In this paper, we have described key micro-architectural and architected features of the POWER9 processor core. The modular execution slice micro-architecture aggregates execution resource building blocks to support a family of processor chips including an SMT8 core design with significant growth in per-core strength and an SMT4 core providing fine-grained control of thread and core resources, both delivering significant thread and socket performance gains relative to POWER8. POWER9 supports enhanced ISA capabilities for emerging workloads including a new breed of data-centric cognitive and accelerated applications.

Acknowledgments

We thank the tremendous innovation, ingenuity, and efforts put forth by the POWER9 chip and systems development teams and the extended teams across IBM who contributed to this project. We would particularly like to thank the efforts of the core design and verification teams, the performance teams including the core performance modeling team led by Richard Eickemeyer and John Griswell, and the performance analysis/verification team. We would also like to extend a special thank you to the lab validation and exerciser teams.

References

1. S. Sadasivam, B. Thompto, R. Kalla, and W. J. Stake, "IBM POWER9 processor architecture," *IEEE Micro*, vol. 37, no. 2, pp. 40–51, May 17, 2017.
2. B. Sinharoy et al., "IBM POWER8 processor core microarchitecture," *IBM J. Res. & Dev.*, vol. 59, no. 1, pp. 2:1–2:2, 2015.
3. IBM POWER ISA version 3.0, Mar. 29, 2017. [Online]. Available: https://openpowerfoundation.org/?resource_lib = power-isa-version-3-0.
4. B. Thompto, "POWER9: Processor for the cognitive era," in *Proc. Hot Chips 28 Symp.*, Aug. 2016, pp. 1–19.
5. T. Rosedahl et al., "Power/performance controlling techniques in OpenPOWER," in *High Performance Computing, ISC High Performance 2017, Lecture Notes in Computer Science (LCNS 10524)*, J. Kunkel, R. Yokota, M. Taufer, and J. Shalf, Eds. Cham, Switzerland: Springer, 2017, pp. 275–289.
6. J. M. Tendler et al., "POWER4 system microarchitecture," *IBM J. Res. & Dev.*, vol. 46, no. 1, pp. 5–25, 2002.
7. B. Sinharoy et al., "POWER5 system microarchitecture," *IBM J. Res. & Dev.*, vol. 49, nos. 4/5, pp. 505–521, 2005.
8. R. Kalla and B. Sinharoy, "POWER7: IBM's next generation POWER microprocessor," in *Proc. Hot Chips 21*, Aug. 2009. [Online]. Available: http://www.hotchips.org/wp-content/uploads/hc_archives/hc21/3_tues/HC21.25.800.ServerSystemsII-Epub/HC21.25.829.Kalla-IBM-POWER7NextGenerationServer Processorv7display.pdf.
9. H. Q. Le et al., "IBM POWER6 microarchitecture," *IBM J. Res. & Dev.,* vol. 51, no. 6, pp. 639–662, 2007.
10. B. Arimilli et al., "IBM POWER9 processor and system features for computing in the cognitive era," *IBM J. Res. & Dev.*, vol. 62, nos. 4/5, Paper 1, 2018.
11. A. Seznec and P. Michaud, "A case for (partially)-tagged geometric history length predictors," *J. Instruction Level Parallelism*, vol. 8, Apr. 2006. [Online]. Available: http://www.jilp.org/vol8.
12. IBM POWER9 NPU team, "Functionality and performance of NVLink with IBM POWER9 processors," *IBM J. Res. & Dev.*, vol. 62, nos. 4/5, Paper 9, 2018.
13. J. Stuecheli et al., "IBM POWER9 opens up a new era of acceleration enablement: OpenCAPI," *IBM J. Res. & Dev.*, vol. 62, nos. 4/5, Paper 8, 2018.

Received April 30, 2018; accepted for publication May 17, 2018

Hung Q. Le *IBM Systems, Austin, TX 78758 USA (hung@us.ibm.com).* Mr. Le is an IBM Fellow in the POWER development team of the IBM Systems group. He joined IBM in 1979 after graduating from Clarkson University with a B.S. degree in electrical and computer engineering. He worked on the development of several IBM mainframe and IBM POWER/PowerPC processors. He was instrumental in introducing state-of-the-art technologies into IBM processors such as advanced high-frequency out-of-order, simultaneous multi-threading, and transactional memory. He is currently developing the next generation of POWER processors. He holds more than 100 U.S. patents in the field of processor design.

James A. Van Norstrand *IBM Systems, Austin, TX 78758 USA (njvan@us.ibm.com).* Mr. Van Norstrand is a Senior Technical Staff Member on the POWER processor team with 36 years at IBM. He graduated from Syracuse University in 1982 with a B.S. degree in electrical engineering from Syracuse University. He has worked during his career on System z mainframe processors, POWER processors from POWER3 through POWER9 as well as on the STI processors chips for gaming systems. For POWER9, he was the processor core lead.

Authorized licensed use limited to: Jeffrey Allen. Downloaded on March 15,2021 at 00:20:01 UTC from IEEE Xplore. Restrictions apply.

Brian W. Thompto *IBM Systems, Austin, TX 78758 USA (bthompto@us.ibm.com).* Mr. Thompto is a Senior Technical Staff Member on the POWER processor development team. He joined IBM in 1999 after graduating from the University of Wisconsin–Madison with a B.S. degree in electrical engineering and a second major in computer science. At IBM, he has been a senior leader and the principal engineer for both IBM POWER and IBM Z processor teams across a number of disciplines and technologies. For POWER9, he was a lead architect and the overall performance lead for the processor chip. Mr. Thompto currently is the chief core architect for the next generation of the POWER processor. He is an IBM Master Inventor with over 50 patent applications granted or pending.

José E. Moreira *IBM Research, Thomas J. Watson Research Center, Yorktown Heights, NY 10598 USA (jmoreira@us.ibm.com).* Dr. Moreira is a Research Staff Member in the Scalable Systems department at the IBM T. J. Watson Research Center. He received a B.S. degree in physics and B.S. and M.S. degrees in electrical engineering from the University of Sao Paulo, Brazil, in 1987, 1988, and 1990, respectively. He also received a Ph.D. degree in electrical engineering from the University of Illinois at Urbana–Champaign in 1995. Since joining IBM at the T. J. Watson Research Center, he has worked on a variety of high-performance computing projects. He was system software architect for the IBM Blue Gene/L supercomputer, for which he received an IBM Corporate Award, and chief architect of the Commercial Scale Out project. He currently leads IBM Research work on the architecture of Power processors. He is author or coauthor of over 100 technical papers and patents. Dr. Moreira is a member of the Institute of Electrical and Electronics Engineers (IEEE) and a Distinguished Scientist of the Association for Computing Machinery.

Dung Q. Nguyen *IBM Systems, Austin, TX 78758 USA (dqnguyen@us.ibm.com).* Mr. Nguyen is a Senior Technical Staff Member in the POWER development team of the Group. He joined IBM in 1986 after graduating from the University of Michigan with an M.S. degree in materials engineering. He has worked on the development of many processors, including POWER3 through POWER9. He is currently the Core Chief Engineer for the next-generation POWER microprocessor. He has more than 100 issued patents and is an IBM Master Inventor.

David Hrusecky *IBM Systems, Austin, TX 78758 USA (hrusecky@us.ibm.com).* Mr. Hrusecky is an advisory engineer in the POWER development team of the Systems and Technology Group. He received a B.S. degree in computer engineering from Rochester Institute of Technology and joined IBM in 1979. At IBM, Mr. Hrusecky has contributed to chip designs in IBM 9370 floating point, vector processing, RAID, MPEG, and now POWER processors. He has worked on the development of Level 1 caches of multiple POWER processor cores from POWER6 through POWER9.

Michael J. Genden *IBM Systems, Austin, TX 78758 USA (mgenden@us.ibm.com).* Mr. Genden is a processor core microarchitect and design lead for IBM Cognitive Systems in Austin, TX, owning delivery of the Instruction Fetch Unit for POWER9. He has a B.S. and M.S. degree in electrical engineering from the University of Texas at Austin.

Michael Kroener *IBM Systems, Boeblingen 71032, Germany (mkroener@de.ibm.com).* Mr. Kroener received a B.S. degree in communications engineering from Fachhochschule fuer Technik in Esslingen in 1994. He joined IBM 1994 to work on floating-point designs for CMOS microprocessors, where he had several design and verification responsibilities. In 1999 and 2003, he joined the IBM development teams in Poughkeepsie and Austin to lead the floating-point development of the processors for IBM Z and IBM POWER systems. He introduced the decimal-floating-point and decimal-fixed-point units into the latest POWER Core processors and lead the implementation of them. He recently took responsibility in logic design for a FPGA implementation of a high-speed communication board for IBM Q. He is author of 30 patents and several technical articles.

Authorized licensed use limited to: Jeffrey Allen. Downloaded on March 15,2021 at 00:20:01 UTC from IEEE Xplore. Restrictions apply.